

Read Strategies

Read strategies are caching techniques optimized for **fast read operations**. They reduce database queries and improve response time.

Most common read strategies:

None

Cache-Aside (Lazy Loading)

Read-Through

1. Why Read Strategies Matter

Repeated reads directly from database cause:

None

High latency

More DB load

Higher cost

Scalability bottlenecks

Caching read strategies solve this.

2. Cache-Aside (Lazy Loading)

Cache sits **beside** the database.

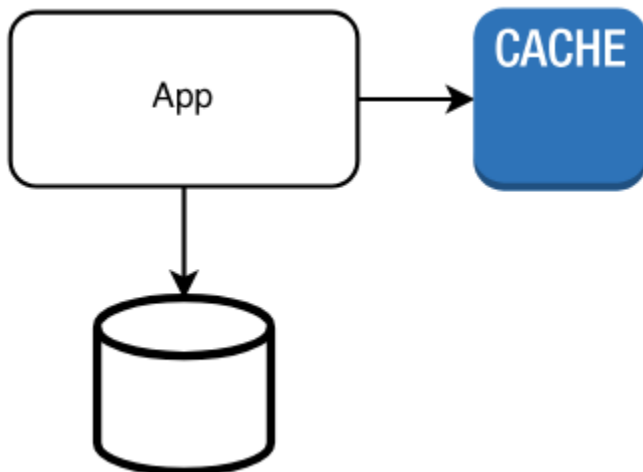
Application handles both:

None

App ↔ Cache

App ↔ Database

3. Cache-Aside Flow



None

User Request



Application checks Cache



Hit → Return data

Miss → Query DB



Store in Cache



Return data

4. Why Called Lazy Loading

Because data enters cache **only when first requested**.

None

No request = no cache entry

5. Example

User opens product page.

None

Cache miss

→ DB fetch product

→ Save in Redis

→ Return response

Next user:

None

Cache hit

→ Instant response

6. Best Use Cases of Cache-Aside

Use for:

- Product pages
- User profiles
- Blog posts
- Search results
- Frequently viewed data

7. Advantages of Cache-Aside

None

Only requested data cached

Simple to implement

Lower memory usage

Flexible cache keys

Can combine multiple DB queries into one cache value

Great for read-heavy traffic

8. Example of Combined Cache Key

Instead of:

None

User query

Orders query

Cart query

Store:

None

dashboard:user_44

One cache read.

9. Disadvantages of Cache-Aside

None

Stale data possible

First request slower (miss)

Cache outage sends all traffic to DB

Extra logic in application

10. Prevent Stale Data

Use:

None

TTL expiry

Invalidate on writes

Write-through strategy

Background refresh

11. Read-Through

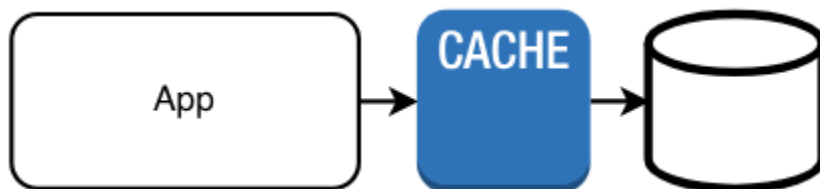
Application talks only to cache.

None

App ↔ Cache ↔ Database

If cache miss, cache itself fetches DB data.

12. Read-Through Flow



None

User Request



App asks Cache



Hit → Return data

Miss → Cache queries DB



Cache stores data



Return data

13. Difference from Cache-Aside

In cache-aside:

None

Application manages DB fetch

In read-through:

None

Cache manages DB fetch

14. Advantages of Read-Through

None

Cleaner application code

App only knows cache

Automatic lazy loading

Good for repeated reads

Centralized caching logic

15. Disadvantages of Read-Through

None

More complex cache layer

Less flexible query composition

May cache unnecessary data

Vendor/tool dependency

16. Cache-Aside vs Read-Through

Feature	Cache-Aside	Read-Through
App accesses DB	Yes	No

App complexity	Higher	Lower
Cache control	Flexible	Centralized
Popularity	Very High	Moderate
Multi-query combine	Yes	Hard

17. HLD Example

None

E-commerce Site

Use Cache-Aside for:

None

Product detail pages

Recommendations

User profile

Use Read-Through when using managed cache systems.

18. Important Failure Scenario

If cache cluster fails:

None

All reads hit DB

Need:

None

Rate limiting

DB replicas

Auto scaling

Fallback mode

19. Interview Answer

If asked “How to optimize read-heavy system?”

None

Use Redis with cache-aside

TTL 5 min

Invalidate on updates

DB replicas for misses

Protect against cache stampede

Strong answer.

20. Advantages Summary

None

Lower latency

Lower DB load

Better throughput

Better scalability

21. One-Line Summary

Cache-aside and read-through are read-optimized caching strategies where frequently accessed data is served from memory, greatly improving performance and reducing database load.